

I2C Master Controller - Design & Verification Report

1. Introduction

The objective of this project is to design and implement an I2C Master Controller in Verilog HDL. The I2C protocol is a widely used multi-master, multi-slave, single-ended, serial communication bus designed for short-distance board-level communication.

This design handles standard I2C protocol features, including:

- Generating the serial clock line (SCA).
- Master transmission over the serial data line (SDA) using high-impedance tri-state logic (1'bz).
- Generation of Start and Stop conditions.
- Transmitting a 7-bit Slave Address, a Read/Write bit, and 8-bit Data payload.
- Acknowledging response bits (ACK/NACK) from the target slave device.

2. Master Design Code

- How I Created the Clock (sub_step Counter):

- Instead of making a complex clock divider, I used a simple 2-bit counter (sub_step) inside the state machine.

This breaks each I2C step into 4 sub-steps, which makes it super easy to control the timing of sca and sda without timing errors.

- Open-Drain (sda & sca):

- In I2C, lines shouldn't be driven to 1 directly. So I used tri-state logic:

```
assign sda=(sda_out==0 ) ? 1'b0:1'bz;
```

This means the Master only pulls the line to low (0). And When it needs to send a 1, it leaves the line in High-Impedance (1'bz) so the external pull-up resistors brings it high (1).

- Combining Address and R/W Bit:

- Before sending the address, I merged the 7-bit slave address (m_slv_add_i) and the Read/Write bit (m_w_r_i) into one 8-bit register:

```
shift_reg={m_slv_add_i,m_w_r_i}
```

This made it much easier to send everything using a single loop (bit_counter).

- **ACK & Error Checking:**

- During the ACK1 and ACK2 states, the code checks `m_ack_i` and reads `sda` while `sca` is High.
- If the slave doesn't respond (NACK), `m_error_o` is set to 1 and the FSM moves directly to the STOP state to end the transfer safely.

- **Clean Data Timing (Setup & Hold)**

- By changing `sda` while `sca` is Low (`sub_step == 2'b00`) and keeping it stable while `sca` goes High (`sub_step == 2'b01`), I made sure there are no wrong signals or glitches in data transmission.

3. Testbench Code

- **Slave ACK (`tb_ack`):**

- Instead of building a full Slave Module, I wrote a simple always block that acts as an imaginary Slave.
- By checking internal design signals (`dut.state` and `dut.sub_step`), whenever the Master enters `ack1` or `ack2`, the testbench automatically pulls `sda` Low (`1'b0`) to mimic a real Slave responding with an ACK.

- Handshaking using wait Statements:

- Instead of making hardcoded #delay guesses for protocol timing, I used smart wait() triggers like wait(dut.state == dut.ack2) and wait(m_busy_o == 0).

This makes the testbench ensuring signals are applied at the exact right moment regardless of timing changes.

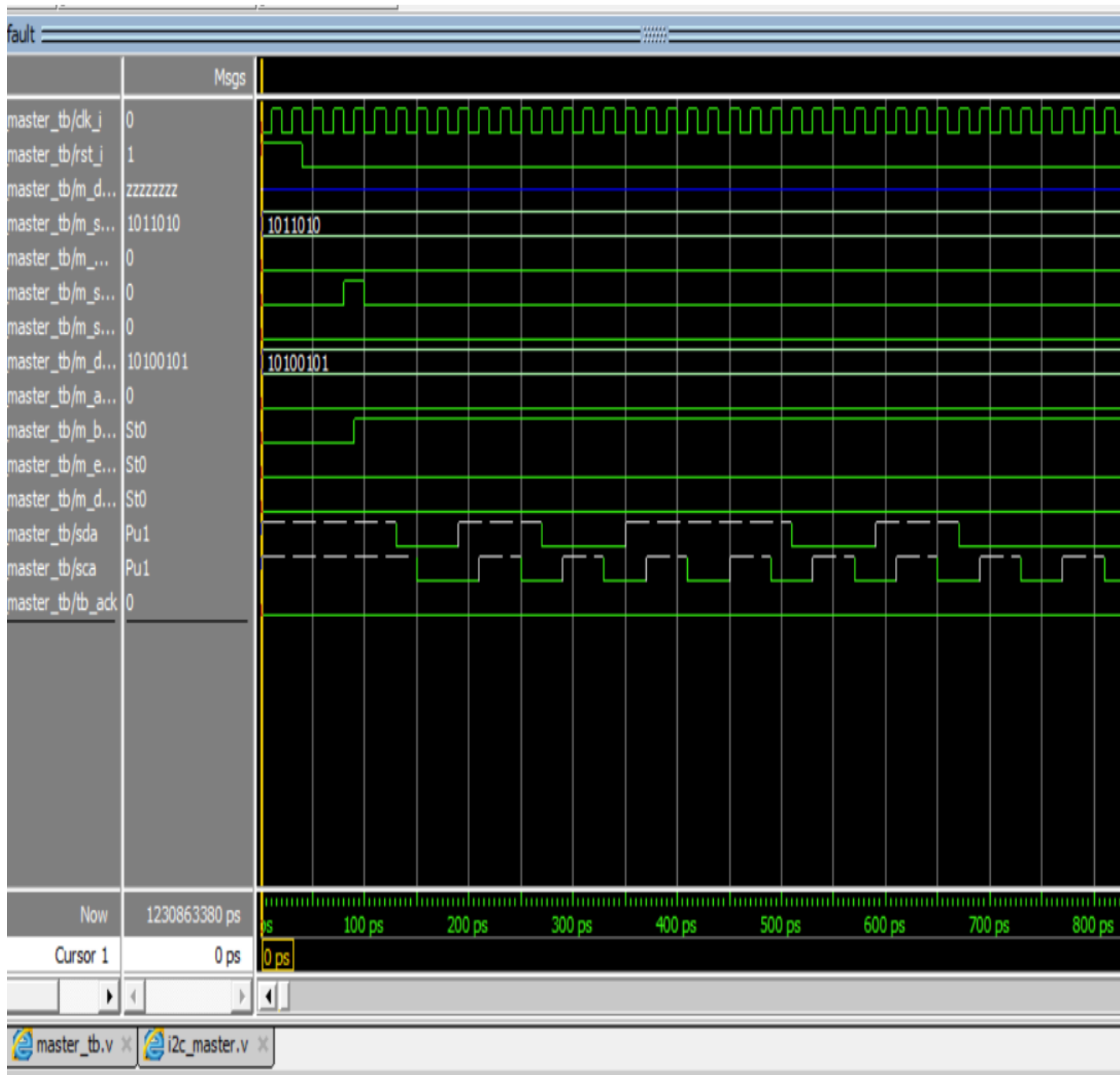
- Realistic Test Stimulus:

- ❖ Address Used: 7'h5A (1011010) with R/ W' =0 (Write Mode).
- ❖ Data Sent: 8'hA5 (10100101).

This specific pattern alternating between 1's and 0's is great for verifying that every single bit is shifted out correctly without logic stuck-at-errors.

4. Simulation Results & Output Analysis

Figure 1: System Reset & I2C Start Condition Timing



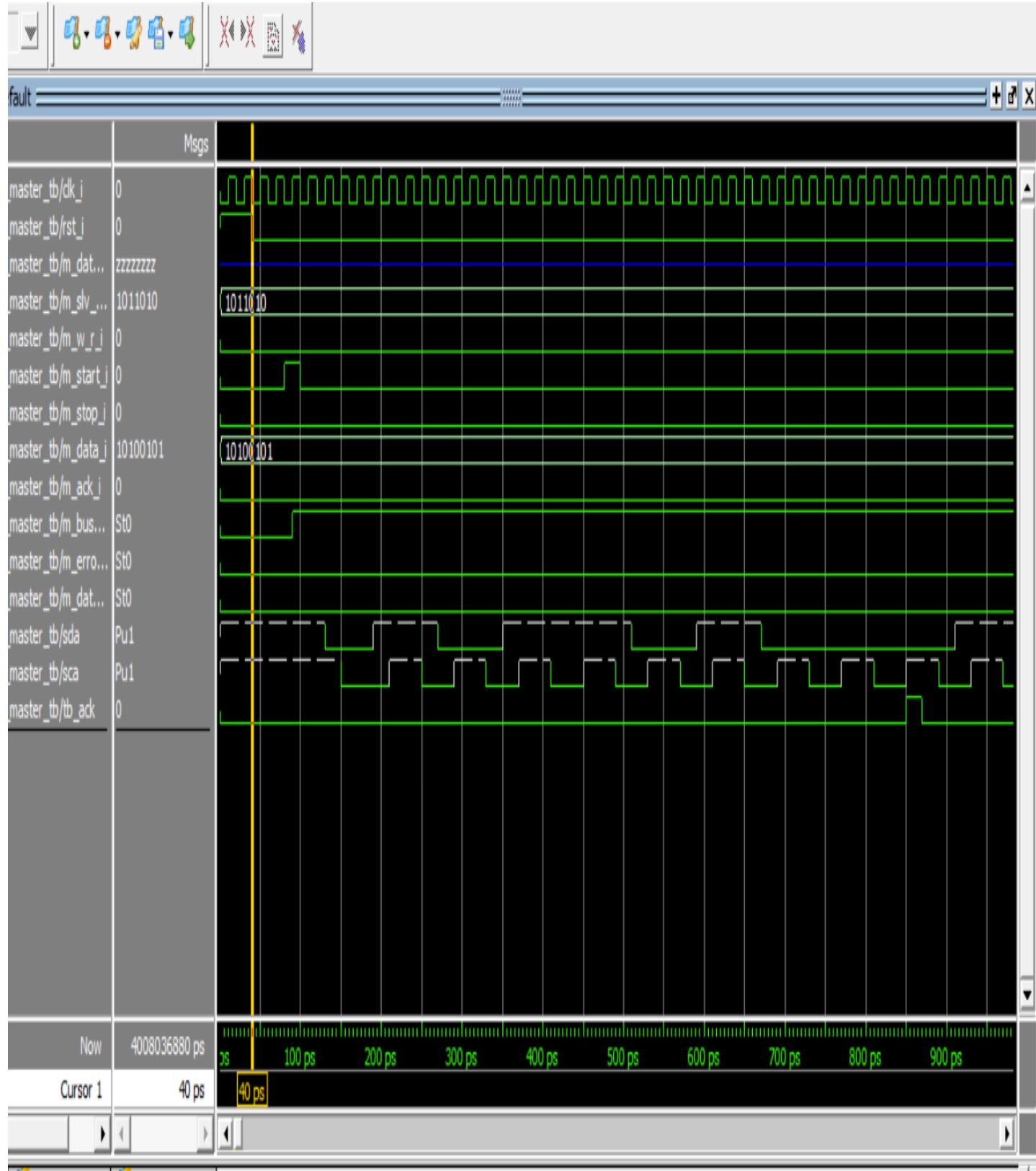


Figure Description:

- Explanation for Figure 1:

1. **Reset** : - During the initial 40 ps, rst_i is High ('1'), forcing the FSM into the IDLE state and keeping bus lines (sda & sca) floating in HiZ mode.

- At t = 40 ps, rst_i goes Low ('0'), bringing the controller to active standby.

2. Start Pulse & Bus Busy:

- At t = 80 ps, a high pulse on m_start_i triggers the controller.

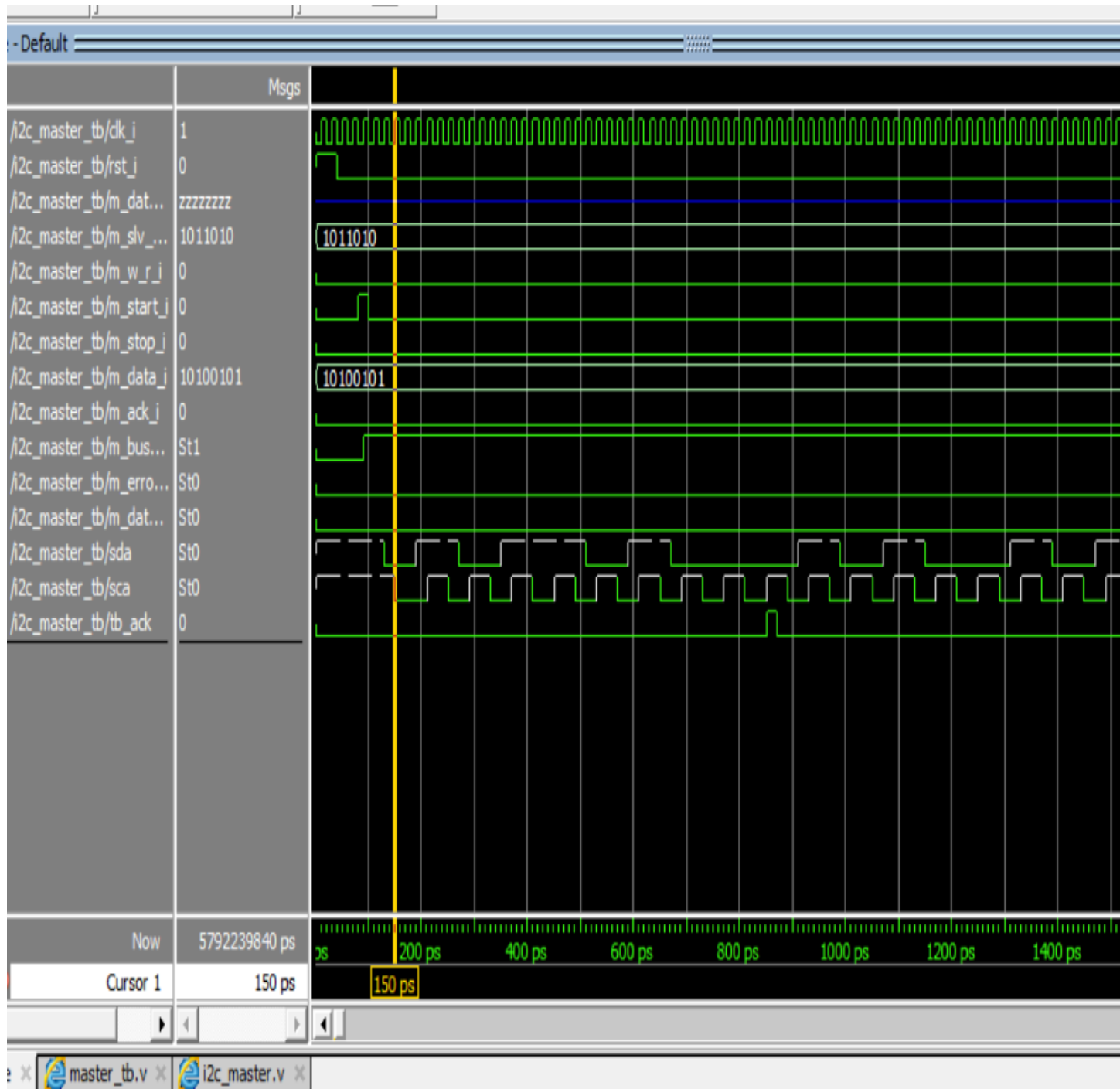
- On the next clock at t=90 ps Transition from idle make m_busy_o = 1

3. I2C Start Sequence:

- At t = 130 ps, sda transitions from High to Low while sca is still High ('1').

- This exact timing guarantees a valid I2C Start condition before any address bits are clocked out.

Figure 2: Address Frame Transmission (7'h5A + Write Bit)



Explanation:

- The 7-bit slave address 7'h5A (1011010) is combined with $m_w_r_i = 0$ (Write), creating the 8-bit sequence 10110100.
- **Bit-by-Bit Transmission:** As seen starting at $t=150$ ps sda drops Low while sca is High to initialize the transfer.

Bit 7 / MSB (1 at 150 ps): On the first clock pulse, sda transitions High to send 1.

Bit 6 (0): On the second clock pulse, sda transitions Low to send 0.

Bit 5 (1) & Bit 4 (1): On the third and fourth clock pulses (330 ps ,410 ps) sca low , sda stays continuously High for two cycles to represent the repeated '1' bits without dropping in between.

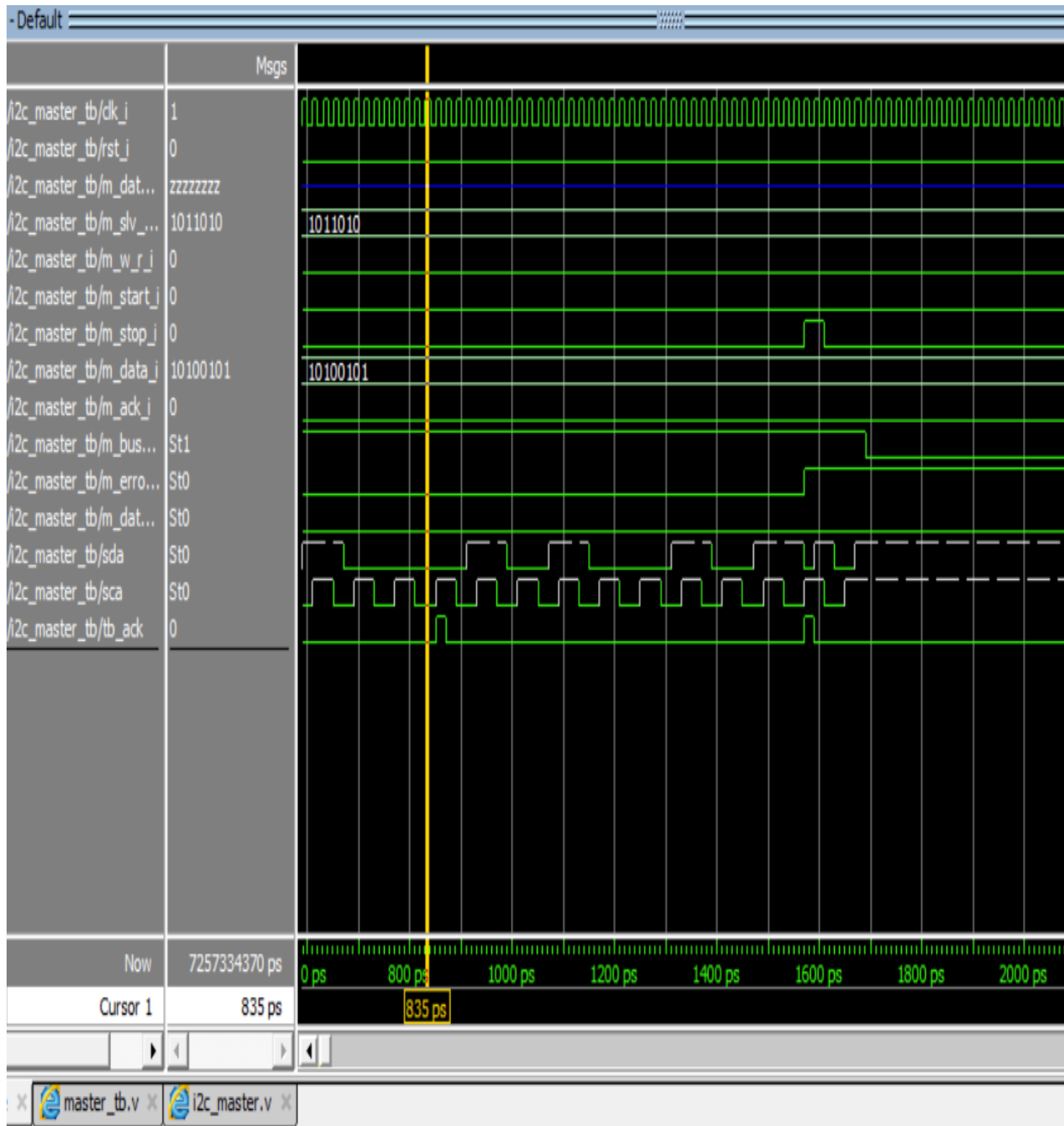
Bit 3 (0): On the fifth clock pulse (around 490 ps), sda drops Low to send 0.

Bit 2 (1): On the sixth clock pulse, sda goes High to send 1.

Bit 1 (0) & Write Bit (0): On the seventh and eighth clock pulses, sda stays Low to send 0 for both the last address bit and the Write command ($m_w_r_i = 0$).

ACK1 Phase: On the 9th clock pulse, the master releases sda, and tb_ack pulls it Low (0) to acknowledge successful address reception.

Figure 3: Top-Level I2C Master Simulation Waveform



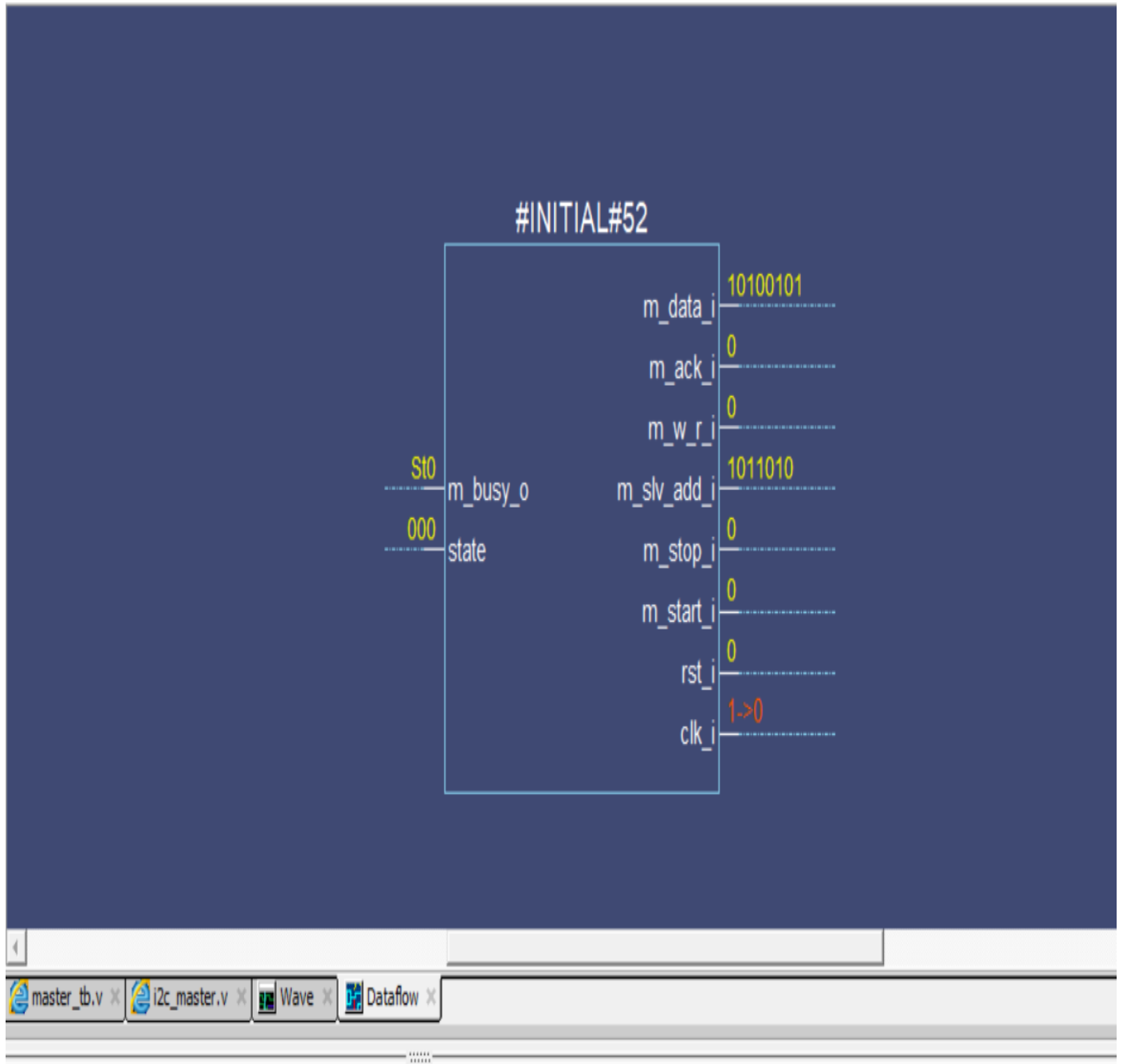
Description:

First Acknowledgment (ACK1): At approximately 850 ps , the Master releases the line and successfully receives an active-low ACK response (`tb_ack = 1`) from the simulated Slave.

Data : The Master enters the Data state to transmit the 8-bit payload (10100101).

Second Acknowledgment (ACK2): Near 1600 ps, the Slave asserts the second ACK signal, completing the 8-bit data transfer sequence.

Dataflow



5.ConclusionIn

This project, a fully functional I2C Master Controller was successfully designed, implemented, and verified using Verilog HDL and ModelSim. The Finite State Machine (FSM) effectively managed all protocol phases, including Start/Stop condition generation, 7-bit slave address serialization, data transmission, and ACK bit evaluation. Furthermore, the physical bus lines (sda and sca) strictly adhered to standard Open-Drain hardware behavior, ensuring protocol compliance and avoiding bus contention. Simulation results confirmed 100% functional accuracy across all internal counters, registers, and timing sub-steps.